

mic — Reading Plans

What the reading plan is, how it's built, validated, shipped, stored, and consumed.

Last updated: August 2026 · App version: 1.0.3+4 · Flutter 3.44.8 / Dart ^3.12.2

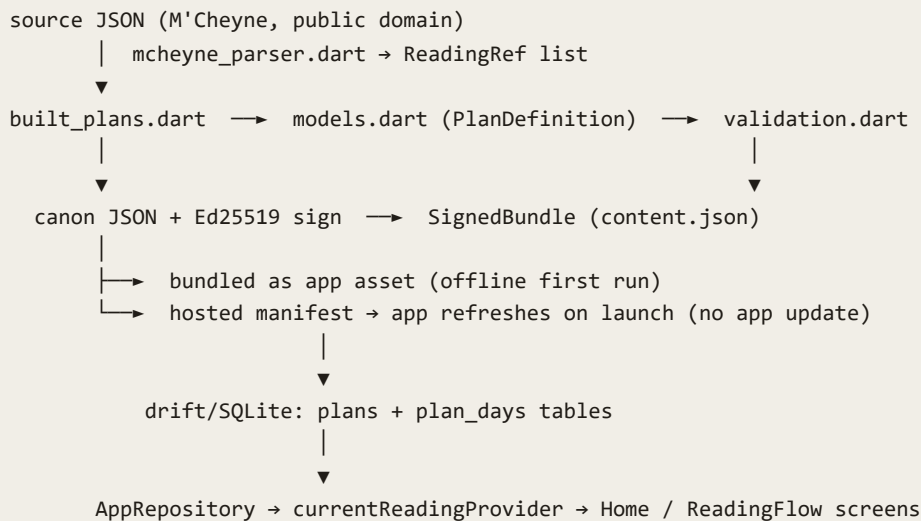
Scope: `content/` (build & sign) · `app/` (storage & UI) · `admin/` (editing)

Contents

1. Overview
2. The three plans
3. Data model (`models.dart`)
4. Canonical book table & time estimates (`book_table.dart`)
5. M'Cheyne source & parser (`mcheyne-plan.json` , `mcheyne_parser.dart`)
6. Building the plans (`built_plans.dart`)
7. Validation rules (`validation.dart`)
8. Signing & distribution (content bundle)
9. In the app: database schema
10. In the app: repository & providers
11. In the app: screens
12. Admin tool
13. Quick reference & FAQ

1. Overview

A reading plan in **mic** is a fixed sequence of *days*, where each day lists one or more *readings* (a book plus a chapter or verse range) and an estimated reading time. Plans are **data, not code**: they are built from source JSON, validated, signed with Ed25519, and shipped to the app inside a versioned *content bundle*. The app stores them in SQLite and drives the "Today" and reading-flow screens from them. Progress is tracked by a stable day index, so updating content never rewrites a user's history.



Everything a user creates (journal, progress, settings) lives only on-device in the app's private SQLite database. Reading-plan content can be updated remotely, but user data is never touched by content updates.

2. The three plans

All three plans are rebuilt from source and shipped together in every bundle.

Slug	Title	Kind	Days	Avg / day	Source
slow-walk	Slow Walk	sequential	1,189	~3 min	Generated — one chapter a day, canonical order.
chronological	Chronological	sequential	1,189	~3 min	Generated — one chapter a day, chronological order.
bible-in-one-year	Bible in One Year	mcheyne	365	~14 min	Parsed — classic M'Cheyne plan, 4 readings/day.

2.1 Slow Walk

`buildSlowWalkPlan()` iterates the canonical `books` table in order (Genesis → Revelation) and emits one `PlanDay` per chapter. Because the 66 books total 1,189 chapters, the plan is exactly 1,189 days. Reading references are always single chapters.

2.2 Chronological

`buildChronologicalPlan()` is identical to Slow Walk except it iterates `chronologicalOrder` — a scholarly, public-domain ordering of the books by the events they describe (Job sits beside Genesis; the prophets are interleaved by era). Still one chapter a day, still 1,189 days.

2.3 Bible in One Year (M'Cheyne)

`buildBibleInOneYearPlan()` reads `source/mcheyne-plan.json` (365 entries, one per calendar day), sorts by month/day, and parses each day's four references — two "Family" and two "Secret" readings drawn from the Old Testament, New Testament, Psalms, and Proverbs. It is the only plan whose readings can be verse-level (e.g. Psalm 119) and the only one with section labels.

3. Data model (`content/lib/src/models.dart`)

Three classes describe a plan end to end:

Class	Fields	Notes
ReadingRef	<code>book</code> , <code>start</code> , <code>end?</code> , <code>section?</code> , <code>startVerse?</code> , <code>endVerse?</code>	One reading reference, e.g. "John 1–3" or "Psalm 119:97–112". <code>end</code> defaults to <code>start</code> for single-chapter readings. <code>section</code> ("Family Reading 1", ...) is only set on M'Cheyne days. <code>display()</code> renders a human string.
PlanDay	<code>day</code> , <code>readings</code> , <code>estimatedMinutes</code>	One day of a plan. <code>day</code> is a 1-based index that stays stable forever — it is the anchor for progress and journal entries.
PlanDefinition	<code>slug</code> , <code>title</code> , <code>description</code> , <code>kind</code> , <code>days</code>	A whole plan. <code>slug</code> is the stable identity (never changes). <code>kind</code> is <code>'sequential'</code> or <code>'mcheyne'</code> . <code>totalDays</code> = <code>days.length</code> .

```
{ "slug": "bible-in-one-year",
  "title": "Bible in One Year",
  "description": "The classic M'Cheyne plan: ...",
```

```
"kind": "mcheyne",
"days": [
  { "day": 1, "estimatedMinutes": 15,
    "readings": [
      { "book": "Genesis", "start": 1 },
      { "book": "Matthew", "start": 1 },
      { "book": "Ezra", "start": 1 },
      { "book": "Acts", "start": 1 } ] },
  ... ] }
```

All three classes have symmetric `fromJson` / `toJson`; the models live in the `pilgrim_content` package so the build tool, the app, and the admin tool share one definition.

4. Canonical book table & time estimates (`book_table.dart`)

- **66 books** in canonical Protestant order, each with `name`, `testament` ("OT"/"NT"), and `chapters`. `totalChapters` = 1,189.
- `bookByName()` resolves canonical names — every reading in every plan must reference a book in this table (enforced by validation).
- `chronologicalOrder`: the 66 books reordered chronologically for the Chronological plan.
- **Time estimates:** `estimateMinutes(book, chapters)` multiplies a per-book minutes-per-chapter rate by the chapter count. The default is `2.5 min/chapter`, with overrides for longer books (e.g. Genesis, Psalms, Isaiah at 3.2–3.5). Estimates are rounded, minimum 1 minute, and stored per day as `estimatedMinutes`.

5. M'Cheyne source & parser

5.1 Source (`content/source/mcheyne-plan.json`)

A public-domain file with 365 entries, one per calendar day:

```
[ { "month": 1, "day": 1, "displayDate": "January 1",
  "readings": [
    { "section": "Family", "label": "Family Reading 1", "reference": "Genesis 1" },
    { "section": "Family", "label": "Family Reading 2", "reference": "Matthew 1" },
    { "section": "Secret", "label": "Secret Reading 1", "reference": "Ezra 1" },
    { "section": "Secret", "label": "Secret Reading 2", "reference": "Acts 1" } ] }, ... ]
```

5.2 Parser (`mcheyne_parser.dart`)

`parseMcHeyneReference()` turns one reference string into one or more `ReadingRef`s:

- Handles chapter ranges (`"Genesis 9-10"`), verse ranges (`"Luke 1:1-38"`), and comma-separated multi-parts (`"Isaiah 8, 9:1-7"`).
- Recognises books longest-name-first (so `"1 Samuel"` isn't read as `"1 Sa..."`); alias `"Psalm"` → `"Psalms"`.
- Merges overlapping or adjacent ranges per book so one day never shows two overlapping readings of the same chapters.

- Verse-level granularity is preserved so long chapters (like Psalm 119, split across many M'Cheyne days) stay true to the original plan.
- The `label` ("Family Reading 1", ...) is carried onto the `ReadingRef.section` field.

6. Building the plans (`built_plans.dart`)

`buildDefaultBundleContent()` assembles all three plans plus the shipped reflection prompts and notification messages into one `BundleContent`. It is shared by the build CLI and the web admin tool so a bundle built in the browser is byte-for-byte consistent with one built on disk.

```
cd content
dart run pilgrim_build build [--version N] [--out FILE]
# 1. reads source/mcheyne-plan.json
# 2. builds the three plans (built_plans.dart)
# 3. validates (validation.dart)
# 4. signs with keys/private_key.pem (Ed25519)
# 5. writes build/content.json AND ../app/assets/content/content.json (the bundled asset)
```

The CLI prints a per-plan day count and verifies the written bundle against the public key before declaring success.

7. Validation rules (`validation.dart`)

Applied before signing; the same rules back the package's automated tests:

- Prompts and notification messages must be non-empty; plan slugs must be unique.
- Every plan must have days numbered exactly `1..N` in order, each day with at least one reading.
- Every reading's book must exist in `book_table.dart`, with chapter bounds within that book and `end ≥ start`.
- **Sequential** plans must cover all 1,189 chapters and read each at most once (full Bible coverage, no repeats).
- **M'Cheyne** plans must be exactly 365 days and cover every chapter at least once — repeats are allowed by design, since the plan interleaves four tracks.

8. Signing & distribution (content bundle)

- Plans ship inside a `SignedBundle`: `{ version, content, signature }`. The signature is an Ed25519 signature over the **canonical JSON** encoding of the content (`canonical_json.dart` keeps the bytes stable).
- The **private** key (`content/keys/private_key.pem`) lives only in the local `content/` folder and is never committed. The **public** key is embedded in the app (`signature_verifier.dart`).
- The app verifies the bundled asset on first run and seeds the DB; on every launch it fetches the remote manifest (`https://sir-mic.github.io/pilgrim/content.json`) and reconciles only if the version is newer. Failures are silent — the app keeps its last-known-good copy and stays usable offline.

- Reconcile upserts plans and days and replaces prompts/messages; progress and journal (keyed by slug + day index, with immutable reading snapshots) are never touched.

9. In the app: database schema

Drift/SQLite (`app/lib/core/database/app_database.dart`, schemaVersion 2, file `pilgrim.db`).

Table	Role for reading plans
<code>plans</code>	One row per plan: <code>slug</code> (PK), <code>title</code> , <code>description</code> , <code>kind</code> , <code>totalDays</code> . Plans are only ever added or updated, never deleted.
<code>plan_days</code>	One row per day, PK <code>(planSlug, dayIndex)</code> : <code>estimatedMinutes</code> and <code>readings</code> (a JSON array of <code>ReadingRef</code> objects).
<code>content_meta</code>	Which bundle <code>version</code> is loaded — gates remote refresh.
<code>sessions</code>	Journal entries: snapshot the day's <code>readings</code> at completion, plus <code>planSlug</code> and <code>dayIndex</code> , so history survives content updates.
<code>reading_progress</code>	Partial "done" indices into today's readings; keyed by <code>(planSlug, date)</code> so it expires daily.
<code>settings_table</code>	Holds the active plan (<code>planSlug</code>) and per-plan completion (<code>progress_<planSlug></code> = days completed this run).

10. In the app: repository & providers

10.1 `AppRepository` (`app/lib/core/data/app_repository.dart`)

- `allPlans()` / `planBySlug()` reconstruct `PlanDefinition`s from the two tables, rehydrating the JSON `readings` array back into `ReadingRef`s.
- `currentReading(planSlug, today)` is the key method: it loads the plan, computes `currentDay = completedCount + 1`, and returns a `CurrentReading` (plan, day, session, partial progress).
- `completedCount()` reads `progress_<slug>` (settings); `totalSessionsForPlan()` counts journal rows; `completedPlans()` finds plans whose max journal `dayIndex` $\geq$ `totalDays`.
- `restartPlan()` resets the run counter and clears the day's partial progress — the journal is untouched.
- Partial progress: `setReadingProgress()` stores the JSON array of done indices for `(planSlug, dateKey)`; writing an empty list deletes the row.

10.2 Providers (`app/lib/core/providers.dart`)

- `currentReadingProvider` (StreamProvider) — re-emits whenever the sessions stream changes: reads the active `planSlug`, throws `NoPlanException` before onboarding, and computes today's reading.
- `allPlansProvider` — all plans for the plan-selection screens.
- `contentSeededProvider` / `remoteRefreshProvider` — verify + seed the bundle, then silently check for updates.

11. In the app: screens

- **Onboarding** — first run; lists the three plans as `PlanCard` s and saves the chosen `planSlug` .
- **Home / Today** (`home_screen.dart`) — watches `currentReadingProvider` ; shows "`{plan.title}` · Day N of `{totalDays}`", each reading's `display()` , and the day's `estimatedMinutes` .
- **Reading flow** (`reading_flow_screen.dart`) — pray → read → reflect. Toggling a reading persists partial progress immediately; finishing writes a journal `Session` and bumps `progress_<slug>` .
- **Progress** — completed days, sessions per plan, completed plans.
- **Settings** — change / restart the current plan (invalidates `currentReadingProvider`).

12. Admin tool

`admin/lib/src/plans_tab.dart` edits each plan's `title` and `description` (with a "Reset to defaults" action). Day schedules are fixed and shown read-only — the plan days always come from `built_plans.dart` , so the shipped schedule is identical whether built on disk or in the browser. Publishing triggers the build + sign and uploads `content.json` to the GitHub Pages manifest URL.

13. Quick reference & FAQ

Where is the plan data model defined?

`content/lib/src/models.dart` — `ReadingRef` , `PlanDay` , `PlanDefinition` , `BundleContent` , `SignedBundle` .

Where are plans actually built?

`content/lib/src/built_plans.dart` — Slow Walk and Chronological are generated from `book_table.dart` ; Bible in One Year is parsed from `source/mcheyne-plan.json` via `mcheyne_parser.dart` .

How are plans stored in the app?

Two drift tables: `plans` and `plan_days` . Days are reconstructed into `PlanDefinition` s by `AppRepository` .

How is the current day decided?

`completedCount(planSlug) + 1` , where completed count is the `progress_<slug>` setting — not the calendar, so you can pause and pick up where you left off.

What happens when content is updated?

Plans/days are upserted, prompts and messages replaced. Progress and journal are untouched because they reference plans by `slug` + stable `dayIndex` and store immutable reading snapshots.

Can a plan be edited by hand?

Only titles and descriptions (admin tool). Day schedules are regenerated from source by `built_plans.dart` and validated by `validation.dart` before signing.

